Setup

Get the non-negotiable decisions right once, before a second contributor joins

Check your engine

Confirm dbt v1 vs v2 before anything else, install story differs

```
dbt --version                          Confirms which engine
pip install dbt-snowflake              v1: adapter is pinned
dbt init my_project                    Scaffolds a new project
```

Secrets, never literal

A connection block should never hold a password as plain text

```
my_project:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "{{ env_var('SF_ACCOUNT') }}"
      user: "{{ env_var('SF_USER') }}"
      password: "{{ env_var('SF_PASSWORD') }}"
      role: transformer
```

Schema strategy

One macro turns three target names into three real environments

```
{% macro generate_schema_name(
    custom_schema_name, node
) %}
  {%- if target.name == 'prod' -%}
    prod
  {%- elif target.name == 'ci' -%}
    ci_pr_{{ env_var('GITHUB_PR_NUMBER') }}
  {%- else -%}
    dev_{{ env_var('DBT_USER') }}
  {%- endif -%}
{% endmacro %}
```

Three environments

dev, ci, and prod stay isolated so builds can't collide

```
dev_{{ env_var('DBT_USER') }}
One schema per developer
ci_pr_{{ env_var('GITHUB_PR_NUMBER') }}
Torn down after the PR
prod
The actual serving schema
```

Git basics

A branch per change, and a commit message that says why

```
git checkout -b feature/x              New branch
git commit -m "Refactor stg_orders"    Say why, not what
git push -u origin feature/x           Push and track
gh pr create                           Open the PR
```

RBAC from day one

Three roles cover every real access pattern, any platform

```
LOADER -> TRANSFORMER -> REPORTER      Role chain
CI/CD service account                  Scoped to its schemas only
Human accounts via SSO                 Interactive dev work
Read-only analytics role               BI tools, never write
```

Version pinning

Pin every version so a minor release can't change behavior under you

```
require-dbt-version: ">=1.8.0"         Pin the engine
packages.yml: exact versions           No ranges for adapters
```

Onboarding

Replace a wiki page with one script a new hire actually runs

```
./bootstrap.sh                         Installs, runs dbt deps, provisions
```

Project layout

Materialize each layer once in dbt_project.yml, not per model

```
models:
  my_project:
    staging:
      +materialized: view
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
```

```
stg_ / int_ / fct_ / dim_              Prefix matches the folder
```

AI Infrastructure

The vocabulary every later lesson's AI segment assumes instead of re-explaining

dbt MCP server

The access layer that lets an agent see a project's metadata

`uvx dbt-mcp`	Starts self-hosted server
`Self-hosted`	Full command access, v1/v2/platform
`Remote: HTTP, platform-hosted`	Consumption only
`Discovery, Admin, Semantic Layer`	What it exposes

AI skills

A markdown file installed into the agent, not a dbt package

`/plugin marketplace add dbt-labs/dbt-agent-skills`	Add the repo
`/plugin install dbt@dbt-agent-marketplace`	Enable it
`npx skills add dbt-labs/dbt-agent-skills --global`	Other agents
`Analytics engineering`	What the repo covers
`Semantic layer work`	What the repo covers
`Platform ops, v1->v2 migration`	What the repo covers

Context engineering

Deciding what an agent gets to see, written as real dbt code

```
packages:
  - package: dbt-labs/dbt_context_engineering
```

`dbt deps`	Installs it
`Chunking without cutting a sentence`	What it's for
`AI calls, spend ceiling, run log`	What it's for
`Vector search, no index required`	What it's for
`Grounding/accuracy tests`	What it's for

Hooks

The agent's own harness enforces this, nobody has to remember

`.claude/settings.local.json`	Where hooks live
`sqlfmt --diff $CLAUDE_FILE_PATH`	PostToolUse: format on save
`Refuse a write outside models/`	PreToolUse: block it
`Lint the file the agent just wrote`	PostToolUse: every time

A good prompt

Structure every request the same way so nothing gets assumed

`Role`	Senior analytics engineer on Snowflake
`Context`	Which model, what must stay true
`Task`	The specific change, named
`Format`	Where the output goes
`Constraints`	Dialect, fixtures, no warehouse cost

Ask Wizard to plan first

Let it explain the blast radius before it touches a file

```
Review my uncommitted changes. Explain which
dbt resources changed, trace downstream impact,
and propose a validation plan. Do not edit yet.
```

Validation tiers

Choose how much checking an agent's work gets, before it runs

`Light`	Syntax, lint, no materialize
`Medium (default)`	+ compiles, builds in dev
`Heavy`	+ dev/prod diff, row counts
`Skip`	No structured validation

Subagents

Split a task too big for one agent, scoped like RBAC

`Planner`	Breaks the task into pieces
`Generator`	Does the work, clean context
`Evaluator`	Checks compile, tests, conventions
`Each gets only its own scope`	Not the whole project

Sources

Treat raw data as a contract: schema, quality, freshness, and enforcement

Sources as a contract

The agreement a staging model is actually built against

```
sources:
  - name: raw
    database: analytics_raw
    schema: public
    tables:
      - name: orders
        loaded_at_field: _loaded_at
        freshness:
          warn_after: {count: 12, period: hour}
          error_after: {count: 24, period: hour}
```

The contract, four parts

Schema, quality, SLA, and the enforcement that makes it real

Schema	What columns, what types
Quality	Idempotent, no duplicates on rerun
SLA	The freshness block above
Enforcement	The gate below, not a wiki page

Staging models

One job only: rename and cast, never drop a row or a column

```
select
    id as order_id,
    customer_id,
    cast(amount_cents as numeric) as amount
from {{ source('raw', 'orders') }}
```

```
materialized: view
Not a table

Never drop a row or a column here
Metadata only
```

Generate with codegen

Reading information_schema beats asking an AI to guess

```
dbt run-operation generate_source
From raw schema

dbt run-operation generate_base_model
Staging skeleton

dbt run-operation generate_model_yaml
Properties file

Review and rename
Don't ship as-is
```

Idempotent loads

A retry or a backfill should never double-count a row

```
Blind append
Duplicates on retry/rerun

Merge on a key (order_id)
Rerun is a no-op

Or a surrogate key
When there's no natural one
```

Freshness config fields

What actually defines the SLA half of the contract

```
warn_after: {count, period}
Soft threshold

error_after: {count, period}
Hard threshold

loaded_at_field
Timestamp column to check

filter
Limit how much gets scanned
```

Source freshness

v2 checks model freshness too, not just sources

```
dbt source freshness
dbt v1, sources only

dbt freshness
dbt v2, sources + models
```

Freshness as a run gate

The enforcement that makes the first three sections real

```
dbt build --select "source_status:fresher+"
Not everything

dbt source freshness && dbt build
Gate, then build
```

Development

Manage the complexity that's left once AI is writing most of the SQL

dbt config

The config block every model configures itself through

```
{{ config(
    materialized='table',
    tags=['finance'],
    meta={'owner': 'finance-team'}
) }}
```

DRY SQL

A repeated block earns a macro; one or two copies don't

3+ repeats	Worth a macro
1-2 repeats	Leave it inline
Comment the why	Not the what

Jinja whitespace control

Keep compiled SQL readable, not a wall of blank lines

```
{%- set methods = ['card', 'cash'] -%}
{% for m in methods -%}
  sum(case when payment_method = '{{ m }}'
    then amount else 0 end) as {{ m }}
{% endfor %}
```

Reusable macro

One version everyone calls, not five hand-rolled copies

```
{% macro cents_to_dollars(col, precision=2) %}
    round({{ col }} / 100.0, {{ precision }})
{% endmacro %}
```

dbt packages

Open source and narrow in scope, lower risk than building it yourself

```
packages:
  - package: dbt-labs/dbt_utils
    version: [">=1.3.0", "<2.0.0"]
```

dbt deps	Installs them

Materializations

The build strategy a model starts with, before it needs to change

view	Recomputed every query
table	Rebuilt in full
incremental	Only new/changed rows
ephemeral	Inlined CTE, never built

Incremental model

Only new or changed rows get processed after the first run

```
{{ config(
    materialized='incremental',
    unique_key='order_id',
    incremental_strategy='merge'
) }}
select * from {{ ref('stg_orders') }}
{% if is_incremental() %}
where updated_at >= (
    select max(updated_at) from {{ this }}
)
{% endif %}
```

Incremental strategy

How the update actually happens underneath unique_key

append	No unique_key, just inserts
merge	Update matched, insert new
delete+insert	Fallback when no merge
insert_overwrite	Replaces whole partitions

Escape hatch

Rebuild from scratch when the incremental logic itself changed

```
dbt run -s fct_orders --full-refresh
```

Rebuild from scratch

Platform configs

Physical tuning on top of the four materializations

cluster_by	Snowflake
partition_by / cluster_by	BigQuery
dist / sort	Redshift
file_format / liquid clustering	Databricks

Documentation

Keep a project's metadata trustworthy at a volume no human reviews by hand

dbt docs The description field and the site it compiles into

`description: "..."`	dbt's own doc primitive
`dbt docs generate`	From manifest + catalog
`dbt docs serve`	Hosts it locally

The dbt Catalog What's actually sitting behind the generated site

`Lineage graph`	Model-to-model dependencies
`Test results`	Pass/fail per model
`Source freshness`	Last loaded, per source

Tags and meta Drive selection and carry ownership, same config block

```
tags: ['finance', 'daily']
meta:
  owner: finance-team
```

`dbt build --select tag:finance`	Only that tag

Doc blocks One description written once, referenced everywhere it's needed

```
{% docs order_total %}
The order's total, after discounts.
{% enddocs %}
```

```
description: "{{ doc('order_total') }}"
```

Reference it

A good vs. a useless description Answer the grain and when to use this

`"The order's total amount"`	Useless, restates the name
`"Order total after discount, before tax, USD"`	Useful: grain + when to use this over a similar column

YAML management One file per model, so two edits never collide

`dbt-osmosis yaml refactor`	Organize + cascade
`dbt-osmosis yaml document`	Documentation only
`dbt-osmosis workbench`	Streamlit, live compile
`Spot-check AI drafts`	Not equal scrutiny on all 200

Ask consumers directly Make documentation a group effort, not a solo chore

`"What columns and descriptions do you need for this dashboard?"`	Ask before you build the pipeline

Semantic Layer One metric, defined once, same number everywhere — dbt platform

```
metrics:
  - name: revenue
    label: "Revenue"
    type: simple
    type_params:
      measure: order_total
```

`One API, every BI tool`	Human docs != AI context
`Query it from any BI tool`	Same number, everywhere

Testing

Invest in tests proportional to where data actually enters your control

Test investment by layer

Heaviest where data enters your control, lighter downstream

Staging	Heaviest, first layer you control
Intermediate	Selective, only join/agg risk
Marts	Business-logic, use-case specific

Singular test

A business rule too specific for a generic test to express

```
select order_id from {{ ref('fct_orders') }}
where order_total < 0
```

Generic tests

Reusable, parameterized, the cheap default for a reason

```
columns:
  - name: order_id
    tests: [not_null, unique]
  - name: status
    tests:
      - accepted_values:
          values: ['open', 'closed']
```

Unit tests

Correctness of the logic itself, tested against fixtures

```
unit_tests:
  - name: test_valid_email
    model: dim_customers
    given:
      - input: ref('stg_customers')
        rows:
          - {email: a@b.com, valid: true}
    expect:
      rows:
        - {is_valid_email_address: true}
```

```
dbt test --select "test_type:unit"
```

Run only these

Run unit tests locally

Skip warehouse compute entirely while iterating

```
export DBT_ENGINE_EXPERIMENTAL_LOCAL_UNIT_TESTS=true
```

Opt in

```
config: {compute: local}
```

Per test, runs on DuckDB

```
Snowflake or BigQuery only
```

Other adapters: remote

Package stack

Three different jobs, not three ways to do one thing

dbt_utils	Primary key testing
dbt_expectations	Richer assertions, seeds
elementary	Anomaly detection, auto tests

Route failures to the owner

A shared channel has no clock running on it

meta.owner tag	Resolves to the right channel
Shared #data-alerts channel	Nobody owns the clock

Test cost tiers

Spend compute where the risk actually is

not_null, unique	Every build
Cross-table reconciliation	Scheduled, prod only

Coverage check

Catches an untested model, human or agent-written alike

```
select m.unique_id
from {{ info_schema('models') }} as m
left join {{ info_schema('tests') }} as t
  on t.model_unique_id = m.unique_id
where m.access = 'public' and t.unique_id is null
```

dbt check	Runs it

Governance

Ownership, contracts, and an audit trail that make a project trustworthy

Ownership

An owner arbitrates and chases a fix, not fixes it personally

```
config:
  meta:
    owner: finance-team
```

```
Resolves to PagerDuty/Opsgenie
```
Survives a departure

Model contracts

A breaking change fails the build, not a dashboard

```
config: {contract: {enforced: true}}
columns:
  - name: order_id
    data_type: text
  - name: order_total
    data_type: numeric
```

`Build fails on mismatch`	Not a silent schema drift

dbt artifacts

Every run's own record, worth ingesting, not overwriting

`manifest.json`	Full project graph
`run_results.json`	What ran, pass/fail, timing
`catalog.json`	Metadata + freshness
`dbt_artifacts package`	Ingests them into the warehouse

Audit logging

Who touched what and when, agent sessions included

`Warehouse query history`	Who ran what, when
`dbt run logs`	What changed, by whom
`Access-grant history`	Often shipped to a SIEM
`Agent sessions count too`	Not just people

Project clean-up

The same info_schema pattern, aimed at orphaned models

```
select m.unique_id
from {{ info_schema('models') }} as m
left join {{ info_schema('edges') }} as e
  on e.parent_unique_id = m.unique_id
where e.child_unique_id is null
```

```
dbt ls --select "models/marts/+"
```
Sanity-check refs

Model certification

A claim worth nothing until a check verifies the bar

```
select m.unique_id
from {{ info_schema('models') }} as m
where (m.meta ->> 'certified') = 'true'
  and m.contract_enforced = false
```

```
Owner + tests + contract + docs
```
The bar it checks

dbt Mesh

The real adoption trigger is org structure, not model count

```
models:
  finance:
    +group: finance
```

```
access: private/protected/public
```
Pick one explicitly

```
Enterprise: cross-project ref()
```
dbt-loom: OSS equivalent

```
~10:1 ratio
```
Warrants an enterprise mesh

```
~5:1 ratio
```
Fits hub-and-spoke instead

Deployment

Run less, and fail safely when something does run

Node selection

Build exactly the slice of the DAG a change actually touches

`dbt build --select "+fct_orders"`	+ upstream
`dbt build --select "fct_orders+"`	+ downstream
`dbt build --select "@fct_orders"`	+ up, down, and their up
`dbt build --select tag:finance`	By tag
`selectors.yml`	Reusable, not copy-pasted strings

Retry and clone

Don't rebuild what already succeeded or exists elsewhere

`dbt retry`	Rerun only what failed
`dbt clone --select state:modified+`	Clone changed nodes
`dbt build --select state:modified+`	Build only what changed

dbt Jobs

Each job type exists for a different reason — dbt platform

Deploy	Schedule/API -> prod
CI	PR commit -> ephemeral schema
Merge	Merge -> prod
dbt State	Every run, skip/clone/build
24h run limit	Same cap, every job type

CI jobs in detail

Built to stay cheap no matter the project size — dbt platform

Ephemeral per-PR schema	Torn down after
Runs execute in parallel	Team velocity
Smart cancellation	Newer commit cancels the old run
SQL linting	Starter plan and above

Platform API

Trigger and manage runs from outside the UI — dbt platform

```
curl -H "Authorization: Token $TOKEN" \
  -X POST ".../jobs/<id>/run/"
```

Admin API v2 (legacy) / v3 (current)	Download artifacts, run jobs
Terraform provider	Calls the same endpoints

Pre/post-hooks

SQL that runs around a build, nobody has to remember

```
{{ config(
    post_hook="grant select on {{ this }} to bi_role"
) }}
```

Runs on models, seeds, snapshots	Not sources

dbt State

Rebuilds only what actually changed — dbt platform

`dbt build --manage-state`	Starts a free trial
`dbt state explain`	v2: why skip/clone/build
`dbt-state explain`	v1 (hyphenated)
`config: {state: {lag_tolerance: ...}}`	Per-model freshness window

dbt Checks

Project-quality rules, enforced before a model compiles

```
select unique_id from {{ info_schema('models') }}
where description = ''
```

`dbt check`	Runs all checks
`config: {severity: warn}`	Default is error
`dbt build --skip-checks`	Bypass, on purpose

CI/CD with GitHub Actions

The self-hosted version of the same CI discipline

```
run: >
  dbt build --select "state:modified+"
  --defer --state ./prod-artifacts
env:
  SF_PASSWORD: ${{ secrets.SF_PASSWORD }}
```